Equipo: Qubits

Diego Levy Absalon Navarro Hernandez

Julio Cesar Estrada Flores

Esther Clemente Jiménez

Dixie Yamile Aguilar Cervantes

Carrera: Ingeniería en computación

Materia: Programación de Sistemas Avanzados

NRC: 227458

Clave: IL377

Sección: D01

Profesor: Jorge Ernesto Lopez Arce Delgado

Introducción

En este reporte se documentan los avances de la primera actividad de la materia *Programación de Sistemas Avanzados*, la cual consiste en implementar un algoritmo paralelizable mediante computación distribuida.
Para lograr lo anterior, hay que levantar una red privada virtual (VPN) por medio de *WireGuard*, una herramienta de código abierto que implementa redes privadas virtuales encriptadas. La topología de red implementada es de tipo “estrella”: todos los clientes de la red se conectan a un nodo central, o hub, formando una estrella.
Cada cliente, e incluso el hub, ejecuta cuatro contenedores de *Docker*. Estos contenedores también estarán conectados a la red. Cada contenedor pedirá al hub una tarea; el hub indicará la tarea al contenedor; una vez que el contenedor recibe su tarea, la ejecuta y envía su resultado al hub.
El algoritmo se implementará en el lenguaje de programación Rust, un lenguaje de propósito general muy eficiente y seguro en cuanto a memoria se refiere.
Con el fin de organizar eficientemente las tareas a realizar de la actividad, con ayuda de ClickUp, un *project management softwar*e, se hará uso de dos metodologías ágiles: Scrum y Kanban.

Objetivos

El objetivo de este proyecto es implementar y lograr la paralelización de un algoritmo, programado en lenguaje RUST, mediante la computación distribuida, utilizando un red virtual privada (VPN) configurada con WireGuard. Además, se busca integrar contenedores Docker para la ejecución distribuida de tareas.
Así mismo, se busca la implementación de metodologías ágiles, en este caso, SCrum y Kanban para la organización y estructuración del proyecto.

Desarrollo

Cálculo de pi por método Monte Carlo

La implementación del algoritmo del cálculo de pi por método de Monte Carlo resulta en una prueba definitiva para maximizar el uso del CPU de cada máquina y comprobar el beneficio que contrae el distribuir la carga de trabajo entre los nodos incluidos por consecuencia, no solo se refleja un mejor desempeño en relación a la velocidad de procesamiento, como es el caso del algoritmo Mandelbrot, si no que consigue un mejor resultado en función a los nodos que se añadan (siendo la comprobación de la utilidad de los sistemas distribuidos el propósito principal de esta actividad), logrando así una mayor precisión matemática, valiosísima en el ámbito científico y computacional.

Debido a la naturaleza de este algoritmo no se necesita estar en comunicación constante entre los nodos, ya que la generación aleatoria de números se hace de manera independiente por cada nodo,llevando al máximo el CPU de cada máquina, y posteriormente de que se hayan generado estos datos, ahora sí se procederá con la

comunicación entre nodos para compartir la cantidad de números clasificados como “dentro del círculo” y los números totales generados, para poder realizar la aproximación final.

El cálculo de π mediante el método Monte Carlo se utiliza como un problema de referencia para evaluar el desempeño del sistema distribuido desarrollado. Debido a que el resultado esperado es conocido y el problema es inherentemente paralelizable, permite analizar métricas fundamentales como escalabilidad, balance de carga y eficiencia computacional sin introducir dependencias externas.

El objetivo no es obtener un valor más preciso de π por necesidad práctica, sino utilizar un modelo matemático controlado que represente simulaciones probabilísticas reales empleadas en cómputo científico distribuido.

Planeación y coordinación

Metodología

Para nuestro proyecto elegimos una combinación de dos metodologías: Scrum y Kanban. Tomamos esta decisión porque fue recomendada por dos miembros del equipo, ya que la habían utilizado con anterioridad y mencionan que la combinación de ambas metodologías mejora la organización y la comunicación. Además, la mitad del equipo ya había utilizado previamente esta metodología.

Scrum porque fue la metodología recomendada por el maestro; sin embargo, le hicimos algunas adaptaciones para que se ajustara tanto al tiempo con el que contábamos para esta primera entrega como a nuestras necesidades como equipo y a las necesidades del proyecto.

Kanban fue la metodología que añadimos para tener una mejor visualización tanto del flujo de trabajo como del estado actual del proyecto.

Roles

En el equipo propusimos tres roles para este proyecto, los cuales son:

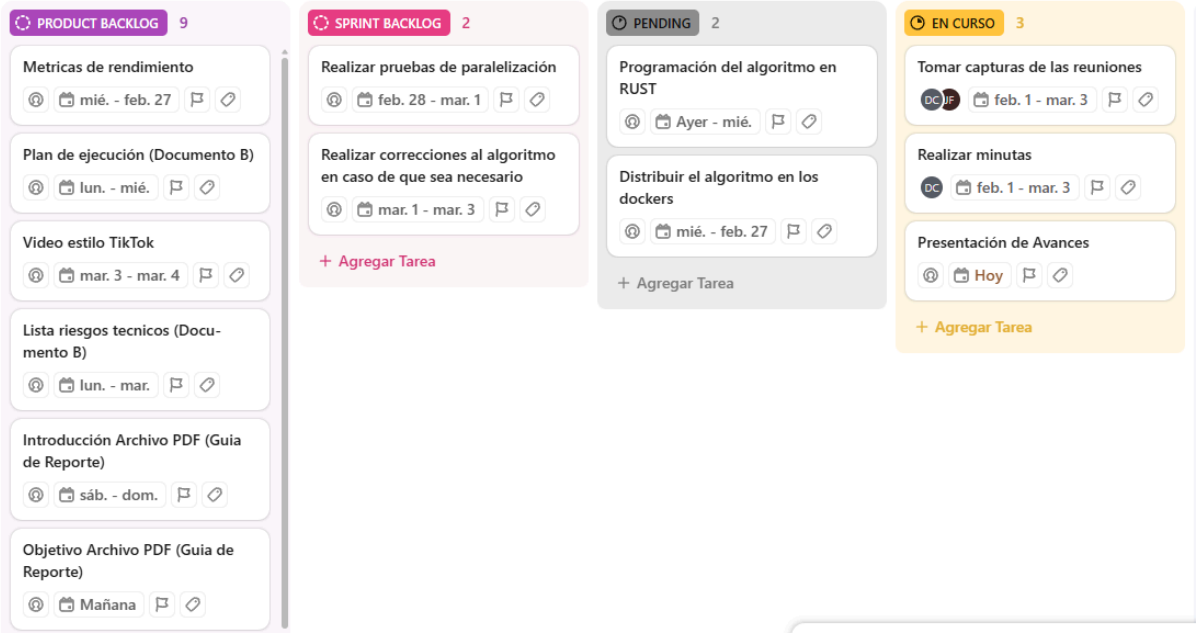
- Scrum Master
- Administrador de red
- Desarrollador

Se tomó la decisión de que todos los miembros del equipo desempeñarán cada uno de los roles propuestos.

Product Backlog

Actualmente, el Product Backlog cuenta con un total de 38 actividades; sin embargo, este número puede incrementarse de acuerdo con los problemas técnicos a los que nos enfrentemos y las decisiones que tengamos que tomar durante el ciclo de vida del proyecto.
En nuestro Product Backlog contamos con actividades como investigación inicial, instalación y configuración de máquinas virtuales, creación de Docker, entre otras.

La gestión del Backlog la realizamos desde la plataforma ClickUp. En el siguiente enlace se puede encontrar el tablero con todas las actividades del Product Backlog y su estado actual: Product Backlog Qubits

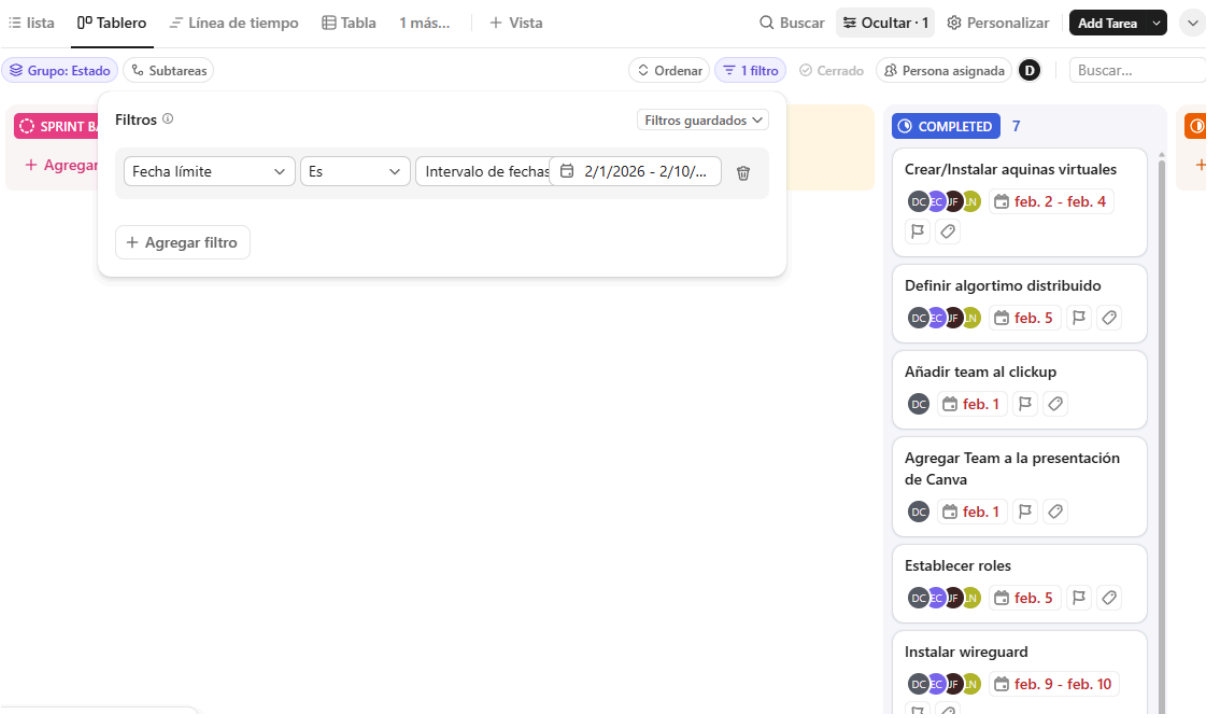


Sprint Backlog

Hasta el momento hemos realizado dos sprints. El primero se llevó a cabo del 01 de febrero al 10 de febrero, y el segundo del 10 de febrero al 18 de febrero.
Dentro del Sprint Backlog del primer sprint tenemos actividades como:

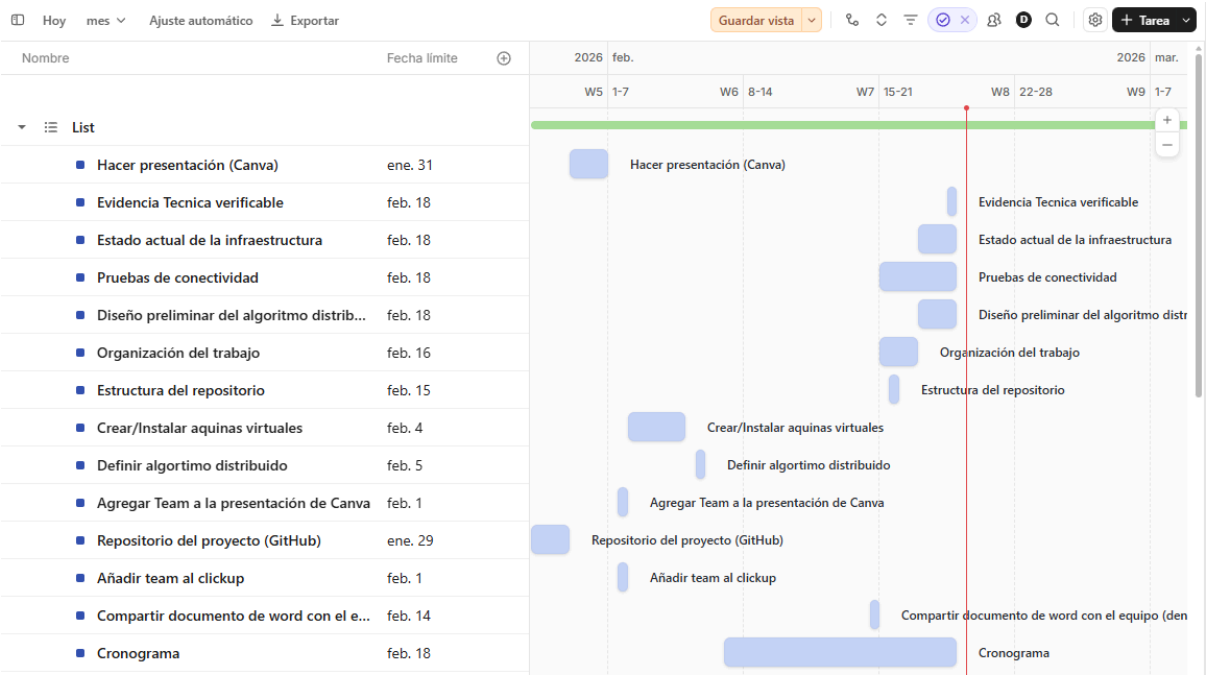
- Entendimiento del proyecto
- Investigación de herramientas y algoritmo
- Instalación de Linux
- Configuración de máquinas virtuales
- Instalación de WireGuard
- Configuración de la VPN
- Crear tablero de ClickUp
- Crear presentación de Canva
- Añadir quipo a la presentación de Canva
- Establecer Roles
- Crear repositorio de GitHub

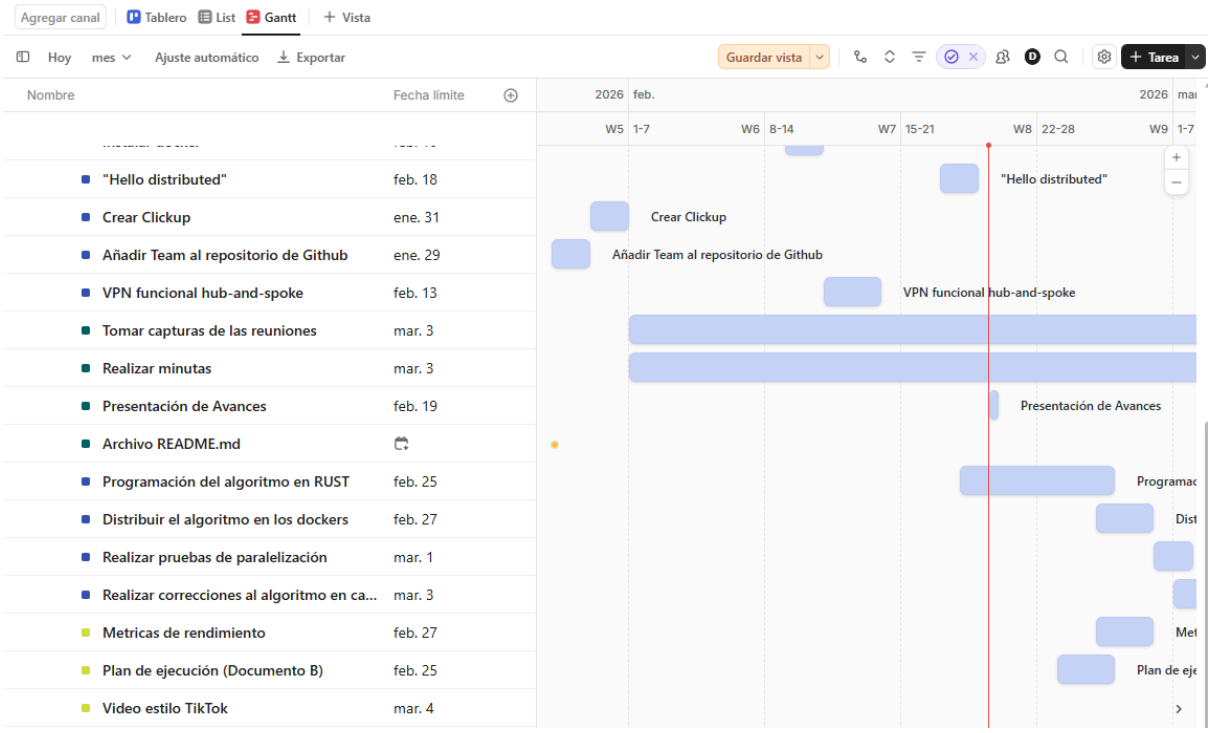
El resto de las actividades se pueden consultar en el siguiente enlace, Product Backlog Qubits, filtrando las actividades que se realizaron entre el 01 de febrero y el 10 de febrero , como se muestra a continuación:



Cronograma

Una de las ventajas de usar [ClickUp]() para la gestión de nuestro Backlog es que, al ir agregando tareas y asignando fechas, la plataforma automáticamente genera un cronograma y un diagrama de Gantt con las actividades.
A continuación, se presentan algunas capturas del cronograma; sin embargo la versión completa y más actualizada se puede consultar en el siguiente enlace: [Diagrama de Gantt Qubits]()





Minutas

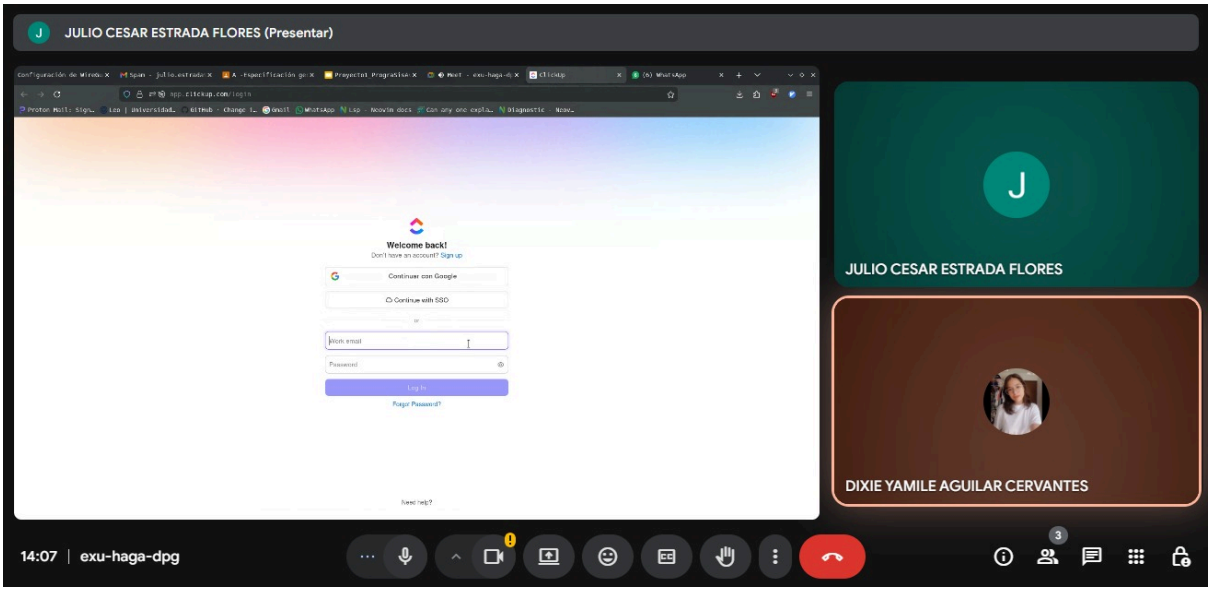
Las minutas de las reuniones son realizadas por un miembro del equipo, el cual va alternando en cada sesión, y se estructuran de la siguiente forma:

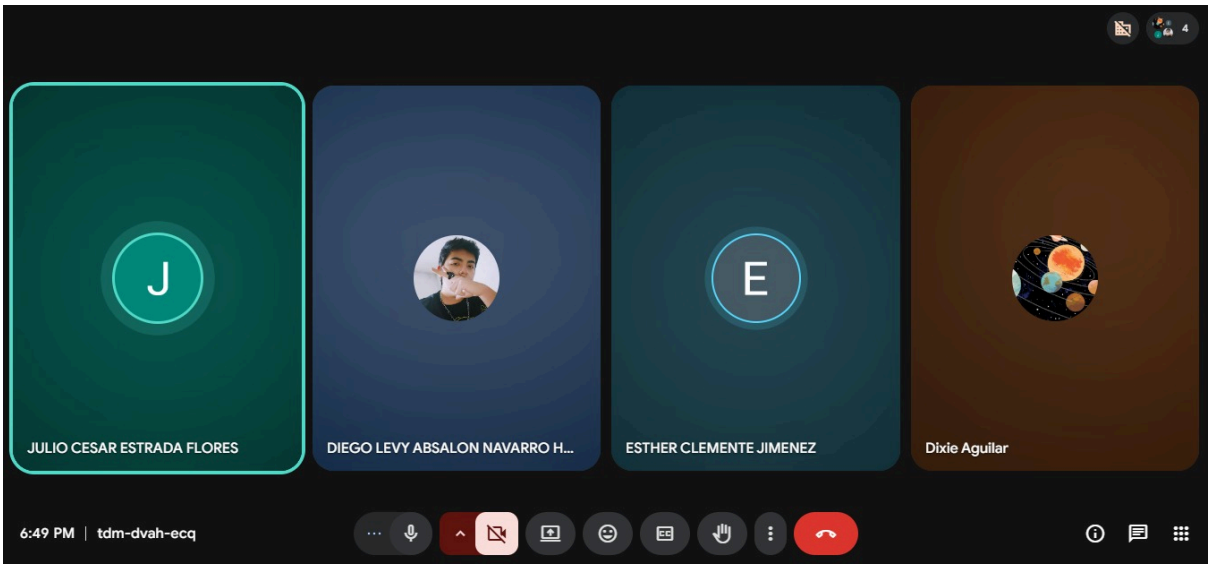
- Fecha
- Objetivo de la reunión
- Actividades realizadas
- Problemas detectados
- Acuerdos
- Próximos pasos

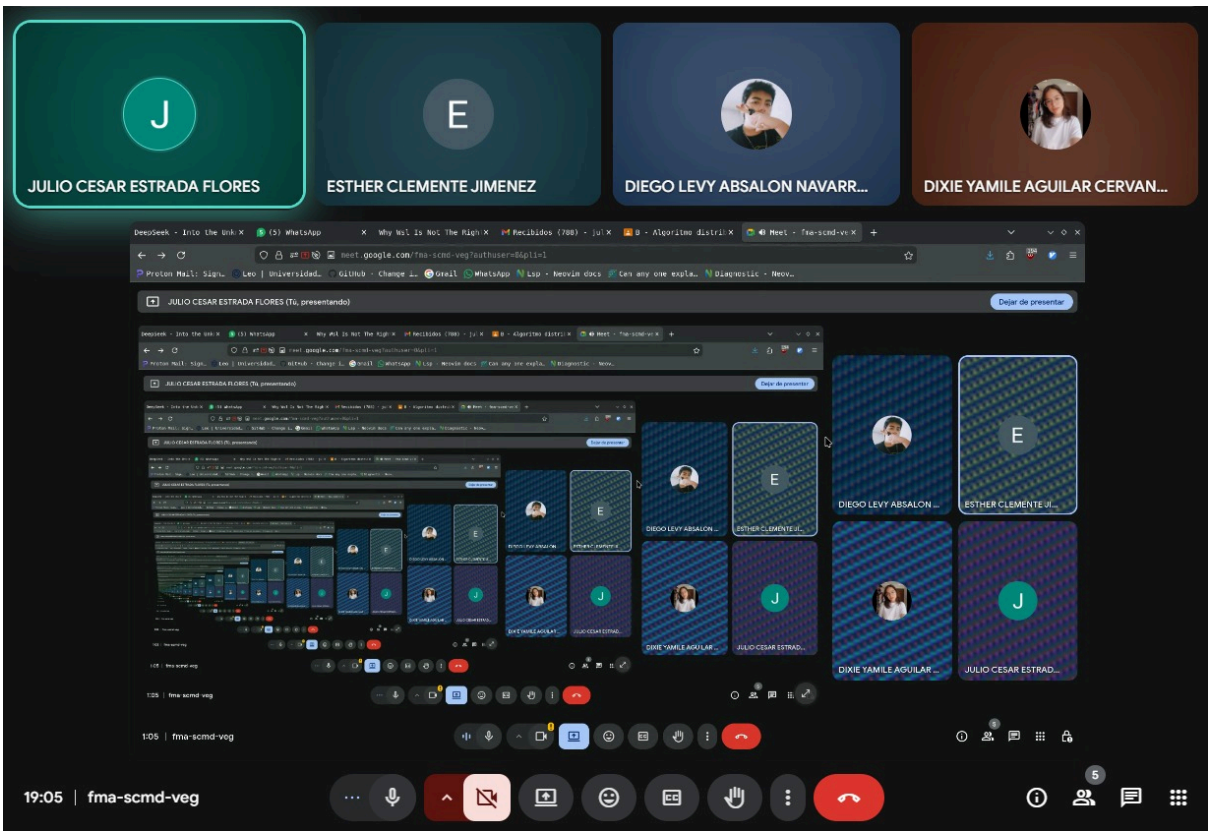
Al final de cada sesión, el miembro del equipo encargado de la minuta pasa toda la información en limpio y después la coloca en el siguiente documento de Google Docs Minutas Reuniones Qubits, en el cual estamos recopilando las minutas de todas nuestras reuniones.

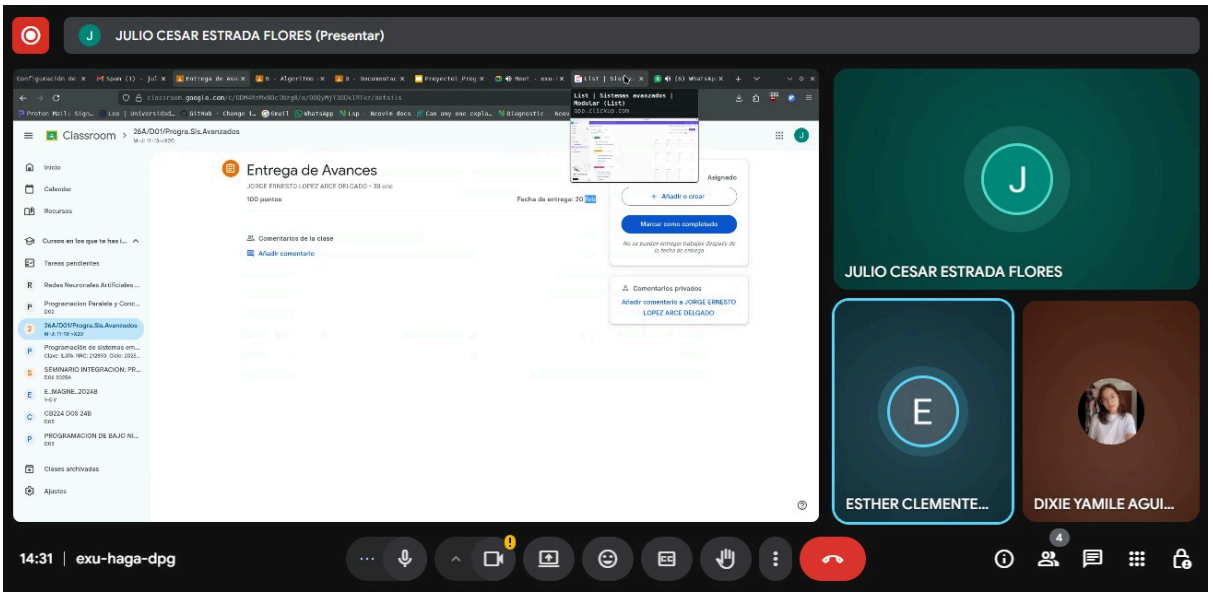
Reuniones

A continuación se presentan algunas capturas tomadas durante nuestras reuniones por Google Meet como evidencia de la comunicación del equipo.


JULIO CESAR ESTRADA FLORES (Presentar)
Welcome back!
Continue con Google
Continue with SSO
Log In
Forgot Password?
Need help?
JULIO CESAR ESTRADA FLORES
DIXIE YAMILE AGUILAR CERVANTES
14:07 | exu-haga-dpg

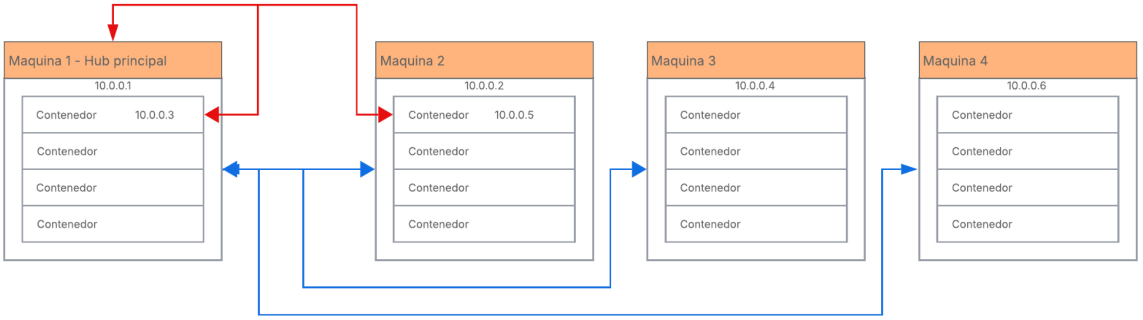

JULIO CESAR ESTRADA FLORES
DIEGO LEVY ABSALON NAVARRO H...
ESTHER CLEMENTE JIMENEZ
Dixie Aguilar
6:49 PM | tdm-dvah-ecq


JULIO CESAR ESTRADA FLORES
ESTHER CLEMENTE JIMENEZ
DIEGO LEVY ABSALON NAVARR...
DIXIE YAMILE AGUILAR CERVAN...
JULIO CESAR ESTRADA FLORES (Tú, presentando)
Dejar de presentar
DIEGO LEVY ABSALON ...
ESTHER CLEMENTE JI...
DIXIE YAMILE AGUILAR ...
JULIO CESAR ESTRAD...
1:05 | fma-scmd-veg
19:05 | fma-scmd-veg


JULIO CESAR ESTRADA FLORES (Presentar)
Classroom
Entrega de Avances
Asignado
+ Añadir o crear
Marcar como completado
Comentarios de la clase
Añadir comentario
Comentarios privados
Añadir comentario a JORGE ERNESTO LOPEZ ARCE DELGADO
JULIO CESAR ESTRADA FLORES
ESTHER CLEMENTE...
DIXIE YAMILE AGUI...
14:31 | exu-haga-dpg

Diseño técnico

Topología

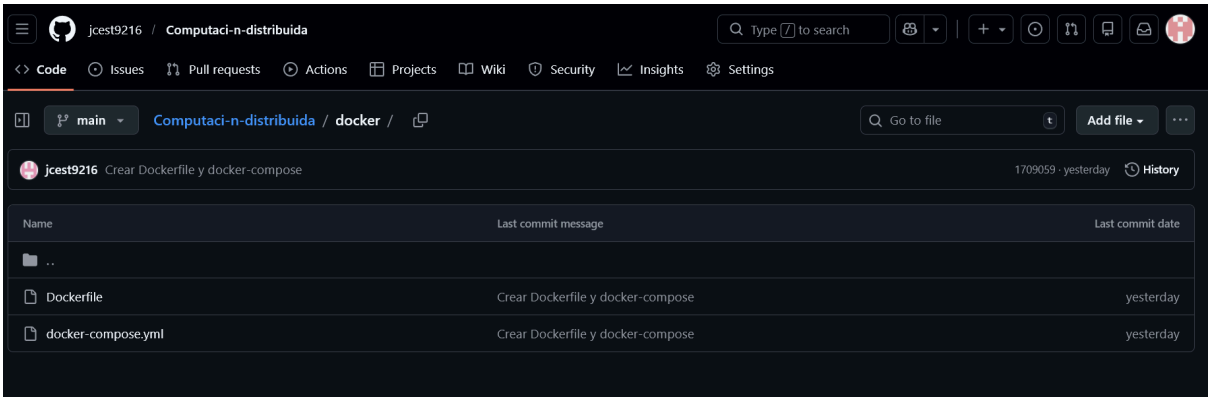


Conexiones: Todas las máquinas son capaces de conectarse mediante la VPN a el hub principal, de la máquina 1 y 2 uno de sus contenedores puede conectarse al hub principal y mandar ping.

Nodos:

- Máquina 1: esta es el hub principal quien se encarga de recibir las peticiones externas.
- Maquina 2, 3 y 4: son los nodos trabajadores, spokes.

Estructura de carpetas



jcest9216 / Computaci-n-distribuida

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

main | Computaci-n-distribuida / rust / | Go to file | Add file

jcest9216 Código de prueba — 493e8f7 · yesterday — History

Name	Last commit message	Last commit date
..		
cliente	Código de prueba	yesterday
servidor	Código de prueba	yesterday
shared	Código de prueba	yesterday
target	Código de prueba	yesterday
Cargo.toml	Código de prueba	yesterday

jcest9216 / Computaci-n-distribuida

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

main | Computaci-n-distribuida / rust / cliente / | Go to file | Add file

jcest9216 Código de prueba — 493e8f7 · yesterday — History

Name	Last commit message	Last commit date
..		
src	Código de prueba	yesterday
target	Código de prueba	yesterday
Cargo.toml	Código de prueba	yesterday

jcest9216 / Computaci-n-distribuida

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

main | Computaci-n-distribuida / rust / servidor / | Go to file | Add file

jcest9216 Código de prueba — 493e8f7 · yesterday — History

Name	Last commit message	Last commit date
..		
src	Código de prueba	yesterday
target	Código de prueba	yesterday
Cargo.toml	Código de prueba	yesterday

jcest9216 / Computaci-n-distribuida

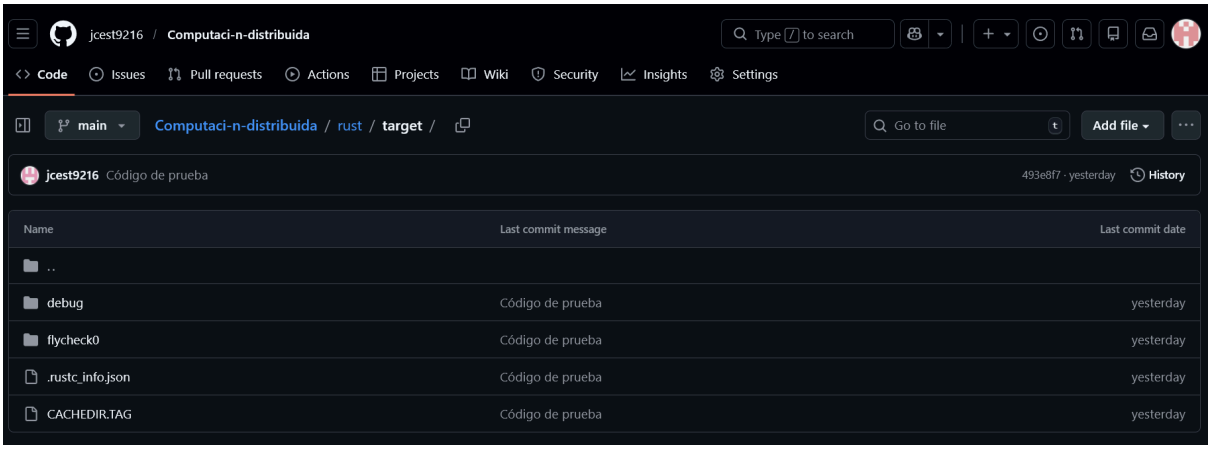
Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

main | Computaci-n-distribuida / rust / shared / | Go to file | Add file

jcest9216 Código de prueba — 493e8f7 · yesterday — History

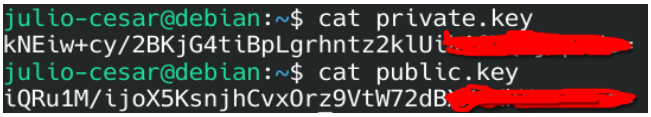
Name	Last commit message	Last commit date
..		
src	Código de prueba	yesterday
Cargo.toml	Código de prueba	yesterday



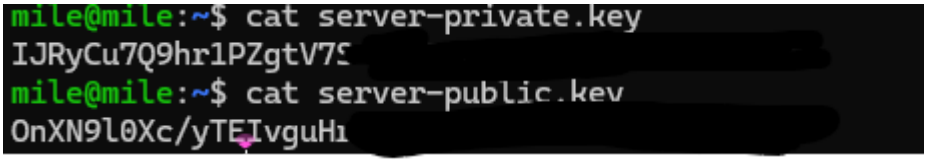
Implementación

Llaves de Wireguard

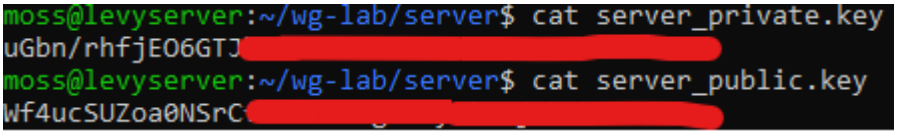
Máquina 1 - Julio Cesar



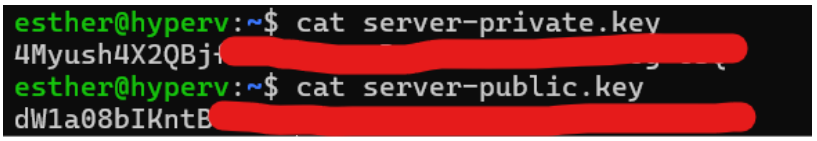
Máquina 2 - Yamile



Máquina 3 - Levy



Máquina 4 - Esther



Evidencia de conectividad

Conexión de Hub al nodo 2 (Yami)

```
julio-cesar@debian:~$ ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=65.0 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=62.0 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=74.0 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=64.3 ms
^C
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 62.038/66.348/74.037/4.573 ms
```

Conexión del nodo 2 (Yami) al Hub

```
mile@mile:~$ ping 10.0.01
PING 10.0.01 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=68.5 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=68.5 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=69.6 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=64.7 ms
^C
--- 10.0.01 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 64.674/67.822/69.584/1.869 ms
```

Evidencia de Docker y Docker Compose

Máquina 1 - Julio (HUB)

```
julio-cesar@debian:~/computacionDistribuida/rust/target/debug$ docker compose version
Docker Compose version v5.0.2
julio-cesar@debian:~/computacionDistribuida/rust/target/debug$ docker --version
Docker version 29.2.1, build a5c7197
```

Máquina 2 - Yami

```
mile@mile:~$ docker compose version
Docker Compose version v5.0.2
mile@mile:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

Máquina 3 - Levy

```
moss@levyserver:~/wg-lab$ docker compose version
Docker Compose version v5.0.2
moss@levyserver:~/wg-lab$ docker --version
Docker version 29.2.1, build a5c7197
```

Máquina 4 - Esther

```
esther@hyperv:~$ docker-compose --version
docker-compose version 1.29.2, build unknown
esther@hyperv:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

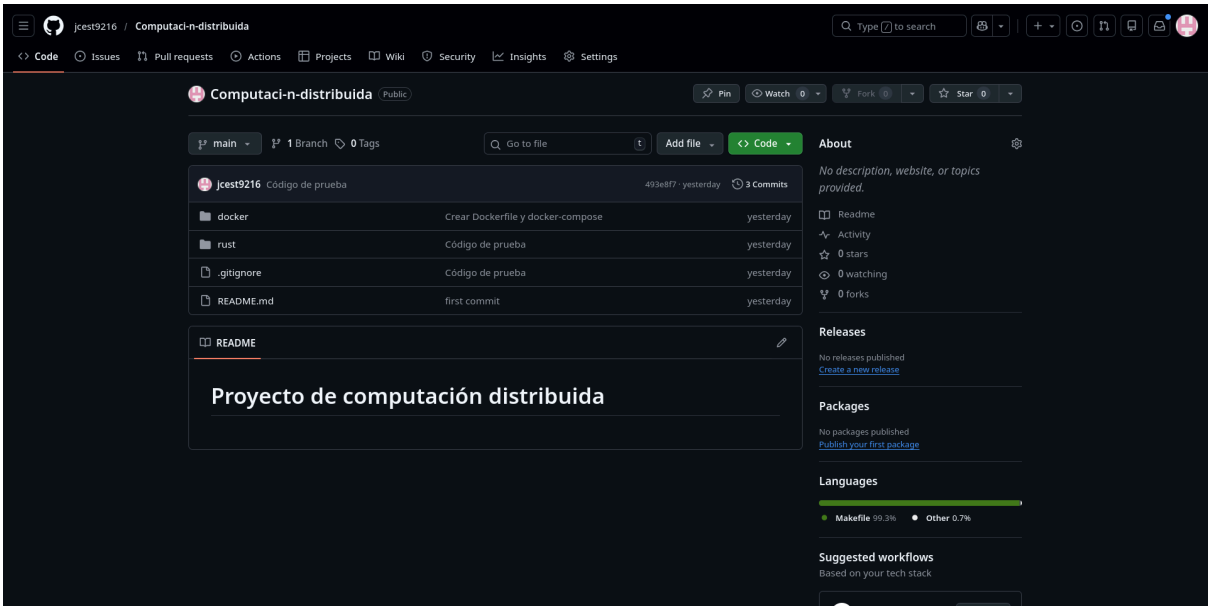
Hello distributed

```
julio-cesar@debian:~/computacionDistribuida/rust/target/debug$ ./servidor
Escuchando en: http://10.0.0.1:3000
Tarea asignada: id=1
Payload: Tarea número 1: procesa este dato
Resultado recibido de la tarea con id 1: procesado: Tarea número 1: procesa este dato
```

```
root@de02a09274c6:~# ./cliente
Tarea recibida: id=1, tarea=Tarea número 1: procesa este dato
Respuesta: Resultado recibido correctamente
```

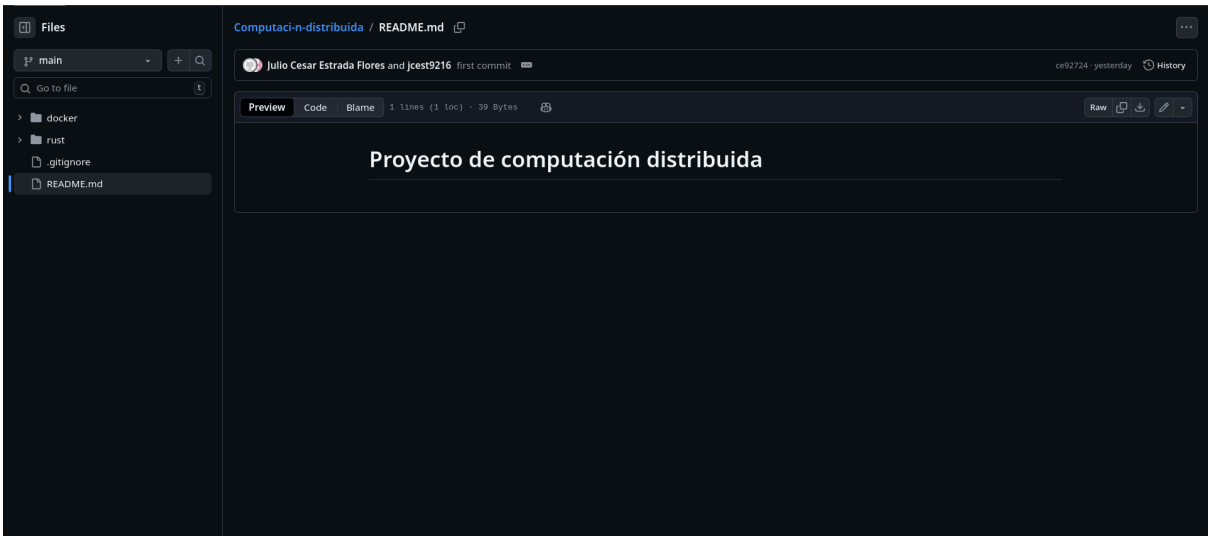
Control de Versiones

Repositorio de GitHub

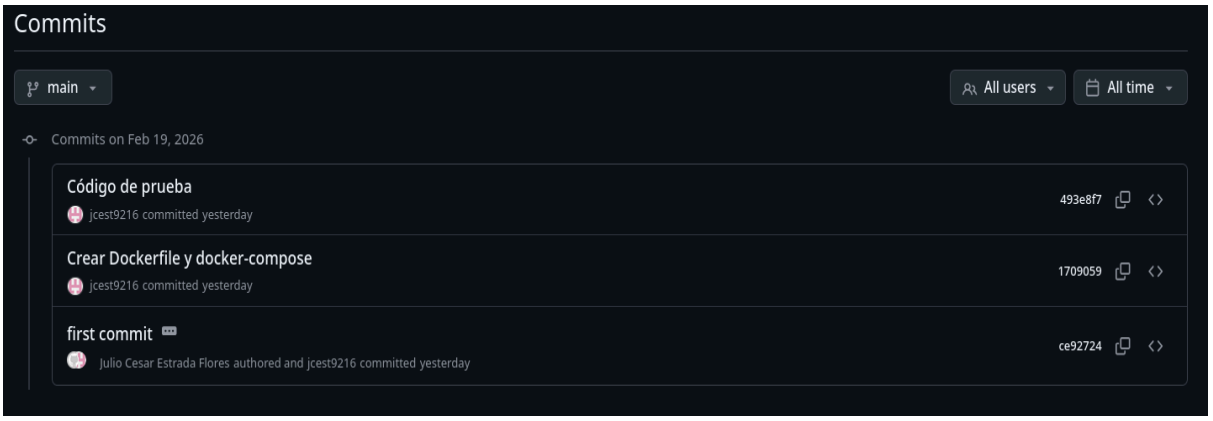


https://github.com/jcest9216/Computaci-n-distribuida

Archivo README



Commits



Reflexión Técnica

Al inicio del proyecto se acordó usar WSL2 sin conocer realmente que este no permite la conexión entre los nodos, no permite ser el hub. Después de varios intentos e investigación continua de la solución a este problema, se buscaron alternativas y al final se llegó a la conclusión de que la opción óptima era usar Hyper V y seguido de eso se hizo la migración a este sistema.

Conclusión

Se logró levantar correctamente la VPN por medio de WireGuard. Tres máquinas virtuales de Linux, y una nativa, se conectaron a la red, es decir, la red está conformada por cuatro nodos, incluyendo el hub. Cada nodo, por medio de Docker Compose y Dockerfile, creó y ejecutó cuatro contenedores, de los cuales sólo uno de la máquina 1 (el HUB), uno de la máquina 2 y uno de la máquina 3 fueron conectados a la red.

Después se verificó la conexión entre los nodos conectados a la red. La conexión fue exitosa. Aunado a esto, se escribió un pequeño programa en Rust. El programa se dividió en dos partes: servidor y cliente. La parte de *servidor* se ejecuta en el nodo HUB. Lo que hace esta parte es “escuchar” si un cliente pide una tarea; de ser así, el servidor envía una tarea al cliente y espera su respuesta. La parte del cliente se ejecuta en los demás nodos. Esta parte pide una tarea al servidor; cuando recibe la tarea, la realiza y, finalmente, envía los resultados al servidor. Este programa funcionó correctamente, lo cual, inferimos, nos dará una base sólida para implementar el algoritmo que seleccionamos, Cálculo de pi por método Monte Carlo.

Referencias

- Ratliff, S. (2026, 29 enero). Docker: Accelerated Container Application Development. Docker. https://www.docker.com/
- Donenfeld, J. A. (s. f.). WireGuard: fast, modern, secure VPN tunnel. https://www.wireguard.com/
- GeeksforGeeks. (2022, 28 julio). Estimating the value of Pi using Monte Carlo. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/estimating-value-pi-using-monte-carlo/